Logistic Regression

Python 3 (ipykernel)

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

In [2]:
```
data = {
    "Hours": [1, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6, 6.5, 7, 7.5, 8, 9, 10],
    "Pass":  [0, 0, 0,   0, 0,   0, 1,   0, 1,   1, 1,   1, 1,   1, 1, 1]
}

df = pd.DataFrame(data)
df
```

Out[ ]:

	Hours	Pass
0	1.0	0
1	2.0	0
2	2.5	0
3	3.0	0
4	3.5	0
5	4.0	0
6	4.5	1
7	5.0	0
8	5.5	1
9	6.0	1
10	6.5	1
11	7.0	1
12	7.5	1
13	8.0	1
14	9.0	1
15	10.0	1

In [3]:
```
X = df[["Hours"]]
y = df["Pass"]
```

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

X_train.shape, X_test.shape
```

Out[ ]:
```
((12, 1), (4, 1))
```

In [5]:
```
model = LogisticRegression()
model.fit(X_train, y_train)
```

In [6]:
```
y_pred = model.predict(X_test)

print("Predicted labels:", y_pred)
print("Actual labels:   ", y_test.to_numpy())
```

Out[ ]:
```
Predicted labels: [0 1 1 1]
Actual labels:    [0 1 0 1]
```

In [7]:
```
print("Accuracy:", round(accuracy_score(y_test, y_pred), 2))
print("Confusion matrix:")
print(confusion_matrix(y_test, y_pred))
```

Out[ ]:
```
Accuracy: 0.75
Confusion matrix:
[[1 1]
 [0 2]]
```

In [8]:
```
new_data = pd.DataFrame({"Hours": [5.0]})
prediction = model.predict(new_data)[0]
probability = model.predict_proba(new_data)[0, 1]

print("Prediction (1 = Pass, 0 = Fail):", prediction)
print("Probability of passing:", round(probability, 2))
```

Out[ ]:
```
Prediction (1 = Pass, 0 = Fail): 1
Probability of passing: 0.71
```

In [9]:
```
hours_grid = pd.DataFrame({"Hours": [i / 10 for i in range(10, 101)]})
probabilities = model.predict_proba(hours_grid)[:, 1]

plt.scatter(X["Hours"], y, label="Actual Pass (1) / Fail (0)")
plt.plot(hours_grid["Hours"], probabilities, label="Predicted pass probability")
plt.xlabel("Study Hours")
plt.ylabel("Pass probability")
plt.title("Logistic Regression")
plt.ylim(-0.05, 1.05)
plt.legend()
plt.grid(True)
plt.show()
```

